> **Intelligence delivered by Giulia** — Local-first AI code intelligence for the BEAM.

Tomato — Analysis Report

Generated at: 2026-04-12 19:00 UTC **Branch:** v0.3 (HEAD `6ba64c8`) **Commit history evaluated:** 11 commits since `origin/main`

---

1. Executive Summary

Metric	Value
Source files	37
Modules	36
Functions (total)	212
Public functions	156
Private functions	56
Types	18
Specs	106
Spec coverage	106 / 156 public (67.9%)
Structs	7
Callbacks	0
API surface ratio	73.6% public
Graph vertices	248
Graph edges	261
Connected components	82
Circular dependencies	**2** — see Section 12
Behaviour fractures	0
Orphan specs	0
Dead-code hits	7 (all false positives — see Section 9)

Verdict: The project is in a healthy state after the v0.3 refactor. The Deploy and Store splits dropped change_risk across the two biggest god modules (Deploy from 250 → 156, Store from 746 → 562), spec coverage sits at a respectable 67.9%, there are zero red-zone modules on the heatmap, and zero behaviour fractures or orphan specs. The single largest remaining gap is `TomatoWeb.GraphLive` **(complexity 98, 14 functions, 29% public)** — the last god module standing in the v0.3 plan, with phases

4b (modal components) and 4c (event handler dispatch) still ahead. Two small-scope circular dependencies (`Demo ↔ Store` , `TomatoWeb ↔ TomatoWeb.Layouts`) are the only P0-class issues flagged.

---

2. Heatmap Zones

Heatmap formula: `centrality*0.30 + complexity*0.25 + (no_test ? 100 : 0)*0.25 + coupling*0.20`. A 25-point penalty applies to any module without a matching `_test.exs` file.

Zone distribution: 0 red / 13 yellow / 23 green

Red zone (score ≥ 60)

None.

Yellow zone (score 30–59)

Module	Score	Complexity	Centrality	Max Coupling	Tests?
TomatoWeb.GraphLive	53	98	0	40	no
Tomato.Graph	49	4	10	10	no
Tomato.Demo	47	3	1	90	no
Tomato.Subgraph	47	11	9	7	no
Tomato.Node	42	4	6	12	no
Tomato.Store.GraphFiles	38	10	1	25	no
TomatoWeb.GraphLive.CanvasComponents	36	40	1	11	no
Tomato.Edge	34	1	4	5	no
Tomato.Deploy.SSH	34	11	3	5	no
TomatoWeb	33	9	4	0	no
Tomato.Store.OODN	30	6	1	8	no
Tomato.OODN	30	1	2	3	no
TomatoWeb.CoreComponents	30	23	1	2	no

Green zone (score < 30)

23 modules — including every v0.3-split submodule that has a test file (`Tomato.Store` , `Store.Mutations` , `Store.Persistence` , `Backend.Flake` , `Walker` , `Deploy` , `Deploy.Config` , `Constraint`) and the error controllers. Notable: `Tomato.Store` sits at score 24 *despite* complexity 69 and 32 functions — the test-file presence drops the floor by 25 points.

Test Coverage Gap Analysis (MANDATORY)

11 of 36 modules (30.6%) have matching `_test.exs` files. The 25 untested modules break down as follows:

Actionable — quick wins for v0.3 polish:

Module	Reason gap exists	Effort
Tomato.Node	Small pure module (machine?/1, new/1)	Trivial — <10 tests
Tomato.OODN	Single new/2 constructor	Trivial
Tomato.Edge	Single new/2 constructor	Trivial
Tomato.Graph	Struct + get/put/root helpers	Trivial
Tomato.Subgraph	Add/remove node/edge + input/output helpers	Small — ~15 tests
Tomato.Store.State	Pure history operations (push/undo/redo)	Small — ~10 tests
Tomato.Store.OODN	4 public functions, all pure	Small
Tomato.Store.GraphFiles	Filesystem ops, mostly tested via StoreTest integration	Medium — needs tmp dir setup
Tomato.Store.Machine	add/3 is already covered indirectly via mutations_test	Low priority — add 1-2 direct tests
Tomato.Demo	Seed procedures, already exercised on every startup	Low priority — integration-tested via boot
Tomato.TemplateLibrary	Pure template data lookups	Trivial
Tomato.Deploy.SFTP	Wraps `:ssh_sftp` — needs a fake SSH connection	Hard — requires integration harness
Tomato.Deploy.SSH	Same as SFTP — the auth-resolution path *is* tested via `Deploy.ConfigTest`	Hard — actual connect/2 flow needs a sacrificial sshd
Tomato.Deploy.Rebuild	`rebuild_command/2` is pure (easy), `apply_config/3` needs a fake conn	Mixed — unit-test the pure half
Tomato.Deploy.Diff	`simple_diff/2` already covered in deploy_test.exs	Already covered — the module is a defdelegate wrapper
TomatoWeb.GraphLive	LiveView — needs `Phoenix.LiveViewTest`	Medium — add render + event assertions
TomatoWeb.GraphLive.CanvasComponents	Pure function components, heex renders	Small — `render_component/2` snapshots

By-design — no unit test expected:

Module	Reason
Tomato.Application	OTP lifecycle — tested implicitly by every other test's GenServer setup
Tomato (namespace stub)	Empty shim module
TomatoWeb	Phoenix `use` macros — tested implicitly by every controller/LiveView
TomatoWeb.Endpoint	Phoenix endpoint — tested via HTTP integration, not unit
TomatoWeb.Router	Phoenix router — tested via request assertions
TomatoWeb.Gettext	Gettext wrapper
TomatoWeb.Layouts	Layout templates — heex snapshots only
TomatoWeb.Telemetry	Metric definitions — data, not logic
TomatoWeb.CoreComponents	Phoenix scaffold — generated components

Summary: the heatmap's yellow-zone inflation is almost entirely driven by the 25-point test penalty. If we took 6–8 hours to test the trivial/small modules (Node, OODN, Edge, Graph, Subgraph, Store.State, Store.OODN, TemplateLibrary), at least 6 yellow-zone modules would drop to green and `Tomato.Node` / `Tomato.Graph` / `Tomato.Subgraph` would become the first modules to combine "high centrality" with "test coverage" — the most valuable kind of protection.

---

3. Top 5 Hubs

Sorted by total degree (fan-in + fan-out).

Module	In-Degree	Out-Degree	Risk Profile
Tomato.Store	2	11	Fan-out monster — thin facade over 11 submodules (Mutations, Persistence, GraphFiles, Machine, OODN, State, Backend.Flake, Graph, Subgraph, Node, Edge); low risk to change because only Demo and GraphLive consume it
Tomato.Graph	10	2	Pure hub — the central data structure; everything reads its fields via pattern matching. Stable interface, touching it propagates widely
Tomato.Subgraph	9	2	Pure hub — second-most-depended-on data structure; invariant-holding but touched by almost every mutation path
TomatoWeb.GraphLive	0	8	Fan-out monster — nothing depends on the LiveView, but it orchestrates 8 project modules directly. High internal complexity but zero blast radius; refactoring is safe
Tomato.Store.Mutations	2	5	Bidirectional hub — consumed by Store + Store.Machine, consumes Graph + Subgraph + Node + Edge + Constraint. The mutation engine; changes need careful testing

---

4. Change Risk (Top 10)

Rank	Module	Score	Key Driver
1	Tomato.Store	562	Function count (32) × fan-out (11) — the dispatch facade
2	TomatoWeb.GraphLive	310	Module complexity (98) — concentrated in `handle_event/3` clauses
3	Tomato.Subgraph	308	Centrality (9 dependents) — any struct shape change cascades
4	Tomato.Demo	297	Max coupling (90 calls to Store) — the seeding entry point
5	Tomato.Store.Mutations	258	Function count (10) × mid-fan-out (5)
6	Tomato.Graph	240	Centrality (10 dependents) — the top struct by fan-in
7	Tomato.Store.Persistence	194	Complexity (30) × fan-in (2) — the JSON decode surface
8	TomatoWeb.GraphLive.CanvasComponents	192	Complexity (40) — freshly extracted, many small function components
9	Tomato.Walker	169	Complexity (21) × fan-out (5) — touches the OODN overlay machinery
10	Tomato.Constraint	168	Complexity (22) — DAG validation via Kahn's algorithm

---

5. God Modules

Module	Functions	Complexity	Score
TomatoWeb.GraphLive	14	98	210
Tomato.Store	32	69	176
TomatoWeb.GraphLive.CanvasComponents	13	40	96
Tomato.Store.Persistence	15	30	81
Tomato.Store.Mutations	10	26	68

Commentary:

- **TomatoWeb.GraphLive** is the last unresolved god module in the v0.3 plan. Phase 4a split out canvas components; phases 4b (modal components) and 4c (event handler dispatch into per-domain handler modules) are still ahead. Complexity 98 is concentrated in the `handle_event/3` clause tower (~50 clauses).
- **Tomato.Store** complexity 69 reflects `handle_call/3`'s dispatch pattern, not algorithmic depth. Each clause is a simple delegation to a pure mutation module. Further complexity reduction would require collapsing clauses into a dispatcher table, which would be a readability regression — leaving as-is is the correct call.
- **GraphLive.CanvasComponents** shows up here because 13 public function components all live in one file; the complexity is distributed across `graph_node/1` (heex + conditional branches) and the pattern-matched style helpers. No further split needed — this is cohesive presentation code.
- **Store.Persistence** carries its 15 functions across the `decode_*` family (decode_graph, decode_nodes, decode_edges, decode_subgraphs, decode_machine, decode_target, decode_oodn_overrides, decode_backend, decode_machine_type, decode_node_type, decode_position, decode_oodn) plus encode/flush/peek helpers. The shape mirrors the JSON schema — by design.
- **Store.Mutations** is the pure functional core. Complexity 26 across 10 functions means ~2.6 per function, which is near-minimal. Good.

Per-function complexity drill-down: Skipped for all 5 god modules — Giulia reports 0 functions above cognitive complexity 5 in any of them. The module complexity is spread thin across many small functions (exactly what we want), not concentrated in a few deeply-nested ones. This is a strong signal that the Store and Deploy splits reached their correct end-state: no single function is cognitively heavy.

---

6. Blast Radius (Top 3 Risk Modules)

Tomato.Store (change_risk rank #1)

Depth 1 (direct upstream — modules this calls): Demo, Edge, Graph, Node, Store.GraphFiles, Store.Machine, Store.Mutations, Store.OODN, Store.Persistence, Store.State, Subgraph Depth 2 (transitive upstream): Constraint, OODN Depth 1 (downstream — modules that call this): Demo, TomatoWeb.GraphLive Depth 2 (downstream): none

Total blast radius: **2 downstream modules** (Demo + GraphLive) Function-level edges: 23 MFA→MFA call edges from `handle_call/3` alone

Cascading hub risk: None — neither Demo nor GraphLive appears in the Top 5 Hubs list with non-trivial fan-in. Demo has fan-in 1, GraphLive has fan-in 0. Modifying `Store`'s public API ripples to at most two consumers.

Call chain example: `Tomato.Store.handle_call/3 → Tomato.Store.Mutations.add_node/3 → Tomato.Subgraph.add_node/2 → Tomato.Graph.put_subgraph/2`. A typical `add_node` request traverses 4 modules (facade → mutation → subgraph → graph), all committed in v0.3.

TomatoWeb.GraphLive (change_risk rank #2)

Depth 1 (upstream — modules this depends on): Tomato.Deploy, Tomato.Graph, Tomato.Store, Tomato.Subgraph, Tomato.TemplateLibrary, Tomato.Walker, TomatoWeb, TomatoWeb.GraphLive.CanvasComponents Depth 2 (transitive upstream): 19 additional modules — essentially the entire `Tomato.*` project tree (Backend.Flake, Constraint, Demo, Deploy submodules, Edge, Node, OODN, Store submodules, CoreComponents, Layouts) Depth 1 downstream: **none**

Total blast radius: **0 downstream modules**

Interpretation: GraphLive is the top of the dependency tree — nothing consumes it. This makes it the *safest* god module to refactor despite its high complexity. Every structural change to `graph_live.ex` is contained to the LiveView itself; tests aren't needed for downstream protection because there are no downstream callers. Phase 4b and 4c can proceed with the same zero-blast-radius guarantee we just used for Phase 4a.

Tomato.Subgraph (change_risk rank #3)

Depth 1 upstream: Edge, Node Depth 2 upstream: *(none — Edge/Node have no further dependencies)* Depth 1 downstream: Constraint, Demo, Graph, Store, Store.Machine, Store.Mutations, Store.Persistence, Walker, TomatoWeb.GraphLive Depth 2 downstream: Store.GraphFiles, Store.OODN, Store.State

Total blast radius: **9 direct + 3 transitive = 12 modules affected** Function-level edges: 1 (`new/1 → Node.new/1`)

Cascading hub risk: HIGH. Five of the 9 direct dependents are themselves in the Top 10 change_risk list (Store, Walker, Store.Mutations, Store.Persistence, Graph). Any shape change to `%Subgraph{}` cascades through the entire mutation + walker + persistence stack. **This is the most load-bearing data structure in the project.** If a v0.4+ refactor needs to change subgraph semantics (e.g., adding a scope tag to align with OODN-scoping plans), it needs a phased migration plan: (1) add the new field with a default, (2) extend readers one module at a time, (3) flip the default. Do not rename or remove fields in a single commit.

---

7. Unprotected Hubs

Module	In-Degree	Spec Coverage	Doc Coverage	Severity
TomatoWeb	4	0%	11%	red

Key insight: 106 specs exist project-wide (67.9% coverage of public functions), but they're heavily concentrated in the business-logic layer — `Tomato.Store.*` (25 specs), `Tomato.Deploy.*`, `Walker`, `Backend.Flake`, `Constraint`. `TomatoWeb` is the Phoenix umbrella module that `use TomatoWeb, :live_view` and friends depend on. It's unprotected but it's also almost entirely framework macros — the "0 specs" is inherited from the Phoenix scaffold. Worth adding docstrings and a moduledoc, not worth adding specs to macro helpers. Downgrading the severity from red to yellow in a human reading.

8. Coupling Analysis (Top 10 Internal Pairs)

Filtered to project-internal coupling only — stdlib calls (Enum, Map, String, File, Process, GenServer, Logger) excluded per convention.

Caller	Callee	Call Count	Distinct Functions
Tomato.Demo	Store	90	9
TomatoWeb.GraphLive	Store	40	23
Tomato.Store.Mutations	Graph	24	4
Tomato.Demo	Tomato.Subgraph	16	2
Tomato.Deploy	SSH	14	3
Tomato.Store.Mutations	Subgraph	10	7
Tomato.Walker	Graph	9	3
Tomato.Store.OODN	Graph	8	1
TomatoWeb.GraphLive	Graph	8	2
Tomato.Store	Mutations	7	7

By-design coupling patterns:

- **Demo ↔ Store (90 calls, 9 functions)**: Demo seeds graphs exclusively through the Store public API. The high call count is a feature — Demo should not bypass the GenServer.
- **GraphLive ↔ Store (40 calls, 23 functions)**: The LiveView exercises nearly the entire Store API (23 of 29 public functions). Expected: the UI is how users mutate state.
- **Store.Mutations ↔ Graph/Subgraph**: The mutation engine's bread and butter. Every `add_node` / `add_edge` / `remove_node` touches both.
- **Store ↔ Mutations (7 calls)**: The `handle_call/3` dispatch layer — exactly one delegation per mutation clause.
- **Deploy ↔ SSH (14 calls, 3 functions)**: Post-split, Deploy calls `SSH.connect`, `SSH.disconnect`, `SSH.exec` — this is the only cross-module coupling that survived the refactor, and it's intentional.

No concerning patterns. The v0.3 splits introduced no new surprise coupling.

9. Dead Code

Module	Function	Line
TomatoWeb	router/0	22
TomatoWeb	channel/0	33
TomatoWeb	controller/0	39
TomatoWeb	live_view/0	51
TomatoWeb	live_component/0	59
TomatoWeb	html/0	67
TomatoWeb.Telemetry	metrics/0	23

Ratio: 7 functions flagged out of 212 total (3.3%). **All 7 are false positives.**

The 6 `TomatoWeb.*/0` functions are the Phoenix `use TomatoWeb, :x` macro callbacks — they're invoked via `unquote(apply(TomatoWeb, which, []))` at compile time when other modules write `use TomatoWeb, :live_view` etc. Giulia's static analysis can't trace the `apply/3` indirection.

`TomatoWeb.Telemetry.metrics/0` is called by the Telemetry supervisor via a registered metric definition — another dynamic-dispatch false positive.

Zero genuinely dead functions. No cleanup work needed.

---

10. Struct Lifecycle

Struct	Defining Module	User Count	Logic Leaks	Leak Count
Tomato.Graph	Tomato.Graph	8	8	8
Tomato.Subgraph	Tomato.Subgraph	5	5	5
Tomato.Node	Tomato.Node	3	3	3
Tomato.Edge	Tomato.Edge	2	2	2
Tomato.Store.State	Tomato.Store	1	1	1
Tomato.OODN	Tomato.Store.Persistence	1	1	1
Tomato.TemplateLibrary	Tomato.TemplateLibrary	0	0	0

Graph (8 users): Store, Store.GraphFiles, Store.Machine, Store.Mutations, Store.OODN, Store.Persistence, Store.State, Walker. Every mutation module and the walker read/write Graph fields via pattern matching — this is the core data structure. In Elixir, field-level pattern matching on internal structs is idiomatic; the compiler enforces struct shape, so any rename or removal is caught at `mix compile` time. **Not a defect** —

this is what a healthy central struct looks like in an Elixir codebase. Contrast with OOP "encapsulation leak" concerns, which don't apply here.

Subgraph (5 users): Constraint (validates topology), Graph (holds the map), Store.Mutations (writes), Store.Persistence (decodes), Walker (traverses). Same story: shared data structure across the core pipeline.

Node (3 users): Mutations, Persistence, Subgraph. Mostly touched at construction and decoding; the walker works through Subgraph which holds nodes.

Store.State (1 user): Only `Tomato.Store` touches its own state struct. **Excellent encapsulation** — the state shape is private to the GenServer facade.

Tomato.TemplateLibrary (0 users): The struct has zero references project-wide. **Potentially unused struct** — `TemplateLibrary.all/0` and `by_category/0` return lists of maps, not `%TemplateLibrary{}` instances. Investigate whether the struct definition is stale; if so, delete it in a cleanup pass (no callers to break).

Commentary: None of the logic-leak counts are bugs. Shared data structures crossing context boundaries (Graph, Subgraph, Node, Edge) are the DAG engine's raw material; pattern matching on them is how functional code composes. The only actionable finding is the possibly-stale `TemplateLibrary` struct.

---

11. Semantic Duplicates

2 clusters at ≥ 92% similarity.

Cluster 1 — `TomatoWeb.CoreComponents.flash/1` ↔ `TomatoWeb.Layouts.flash_group/1` (size 2, 94.1% similarity)

Both are Phoenix scaffold-generated heex function components that render flash messages. They share structural similarity (attr declaration + heex template) but serve different purposes (`flash/1` renders a single message, `flash_group/1` renders the flash map). **Structural similarity, not duplication** — leave as-is.

Cluster 2 — `TomatoWeb.Telemetry.start_link/1`, `init/1`, `metrics/0` (size 3, 92.2% similarity)

Three functions inside the same module — the standard `Telemetry.Supervisor` GenServer boilerplate. They cluster because they all match the `def foo(arg), do: ...` shape with minimal body. **Structural similarity from Elixir GenServer idioms**, not duplication.

Neither cluster represents duplicated logic. Both are expected artifacts of how the semantic-similarity algorithm responds to Elixir's tight function-definition conventions.

---

12. Architecture Health

Check	Status
Circular dependencies	**2 cycles found** — P0
Behaviour integrity	Consistent (0 fractures)
Orphan specs	0
Dead code	7 functions flagged, all false positives

Circular dependencies (P0)

Cycle 1: `Tomato.Demo ↔ Tomato.Store`

- `Tomato.Demo.seed/0` (and `seed_multi/0`, `seed_home/0`) calls `Tomato.Store.new_graph/1`, `Store.add_node/3`, etc. — that's `Demo → Store`.
- `Tomato.Store.handle_continue(:maybe_seed, _)` calls `Tomato.Demo.seed/0` in a spawned task — that's `Store → Demo`.

This is a real cycle in the dependency graph, but it's **benign**: the Store→Demo edge exists only in the startup-seeding handle_continue, and it runs in a spawned process, not synchronously. Still worth breaking for cleanliness — the seeding logic could be extracted to a separate `Tomato.Seeder` module that depends on both Store and Demo, so neither has to know about the other.

Cycle 2: `TomatoWeb ↔ TomatoWeb.Layouts`

- `TomatoWeb.__using__(:live_view)` imports `TomatoWeb.Layouts.app/1`.
- `TomatoWeb.Layouts` uses `TomatoWeb, :html` via `use TomatoWeb, :html`.

This is the **stock Phoenix 1.7+ scaffold cycle** — every generated Phoenix project has it. It's architecturally intentional (the layout module needs the same macro bundle as any other HTML component) but Giulia can't distinguish scaffold from project code. **Safe to ignore.**

Behaviour integrity

Consistent. Zero fractures. `Tomato.Application` implements `Application`, `Tomato.Store` implements `GenServer`, `TomatoWeb.GraphLive` implements `Phoenix.LiveView` — all with complete callback sets.

Orphan specs

Zero. No leftover `@spec` declarations for functions that were renamed or removed.

Dead code

Zero actionable dead code. See Section 9 — all 7 flagged functions are dynamic-dispatch false positives from `use TomatoWeb` macro expansion and Telemetry registration.

13. Runtime Health

No runtime data collected — the v0.3 dev server is running but we didn't open a runtime observation session for this report. For future reports, the relevant endpoints are `GET /api/runtime/pulse`, `GET /api/runtime/hot_spots?path=<path>`, and `GET /api/runtime/top_processes?metric=reductions`.

14. Recommended Actions (Priority Order)

P0 — Blocking

1. **Break the `Demo ↔ Store` cycle.** Extract the startup seeding logic from `Tomato.Store.handle_continue/2` into a new `Tomato.Seeder` module that depends on both Demo and Store. The Store no longer references Demo at all; Seeder is invoked from `Tomato.Application.start/2` after the supervision tree is up. Cycle count drops to 1 (the unavoidable Phoenix scaffold cycle). Expected effort: **~30 minutes**, one commit, no behaviour change.

P1 — High-risk gaps

None. `TomatoWeb` is the only red-severity unprotected hub (see Section 7), but it's Phoenix scaffold code with inherited framework-level macro helpers. Not a real protection gap.

P2 / P3 — Improvement opportunities (limit 3 combined)

2. **P2 — Finish the `TomatoWeb.GraphLive` split.** Phase 4b extracts the 5 modal components (`content_editor`, `oodn_editor`, `template_picker`, `generated_output`, `graph_manager`) into `TomatoWeb.GraphLive.ModalComponents`. Phase 4c extracts the ~50 `handle_event/3` clauses into per-domain handler modules under `TomatoWeb.GraphLive.Handlers.*`. Expected impact: graph_live.ex drops from 1536 → ~400 lines, complexity 98 → < 30, and the last god module leaves the Top 5 list entirely.

3. **P2 — Test the trivial modules** (`Node`, `OODN`, `Edge`, `Graph`, `Subgraph`, `Store.State`, `Store.OODN`, `TemplateLibrary`). 6–8 hours of work. Drops 6+ modules from yellow to green in the heatmap and gives genuine protection to the top three pure-hub structs (Graph, Subgraph, Node). Highest leverage per hour of any remaining work.

4. **P3 — Investigate the unused `Tomato.TemplateLibrary` struct.** Zero users project-wide (Section 10). Either promote it to the public API (return `%TemplateLibrary{}` from `all/0` / `by_category/0` instead of raw maps) or delete the struct definition. Either way, the current state is inconsistent — a struct exists but nobody constructs it.

Deferred (tracked in v0.3 but not report-actionable)

- Sidebar editor for leaf `:target` field
- Scoped OODN panel inside each machine subgraph (editor for `oodn_overrides`)
- Local Nix-fragment validation (`nix-instantiate --parse` on leaves before write)
- Windows dev-server zombie BEAM on shutdown (install `:os.set_signal/2` hook in `Tomato.Application`)

Intelligence delivered by Giulia — D:/Development/GitHub/tomato — 2026-04-12